SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: ITA 0603 COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptrons, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

Assessment Tool Weightage: 20%

QUESTIONS: 15 **Total Marks:25**

Annexure A

SCENARIO-BASED QUESTIONS

Code of Conduct:

I Peta.Harsha Vardhan (192311039) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

Short Answer Test

1. With the help of a neat sketch, explain the different functional blocks present in a machine learning system, including data collection, preprocessing, feature extraction, model training, and evaluation. Illustrate the working of the system by providing suitable examples for both supervised and unsupervised learning approaches, clearly highlighting how labelled and unlabelled data are handled within the learning process. (5 Marks)

2. Design a machine learning system for disaster management to predict events such as floods and earthquakes using both historical and real-time data sources. Explain the selection of an appropriate model, including whether a neural network or any other algorithm is suitable for this task, along with justification. Describe the data preprocessing steps and feature selection techniques required to improve model performance. Further, explain the complete process of training, validating, and evaluating the model, and discuss how such a system can assist authorities in early warning, risk assessment, and effective emergency response. (5 Marks)

3. In the context of neural networks, consider a multilayer perceptron used for binary classification. Explain how the backpropagation algorithm updates the weights in the network during training. Illustrate the process using a simple example by describing the forward pass, error calculation, and backward propagation of errors. Discuss how learning rate and activation functions influence the convergence and accuracy of the model. (5 Marks)

4. In the context of Genetic Algorithms, consider a dataset representing different candidate solutions for maximizing a fitness function based on two parameters: Speed and Accuracy. Each solution is evaluated as either suitable (Yes) or not suitable (No). Using the Genetic Algorithm approach, analyze how the population evolves by applying selection, crossover, and mutation operations. Show the step-by-step improvement of candidate solutions across generations and identify the best optimal solution. (5 Marks)

Dataset:

Example	Speed	Accuracy	Fitness Value	Suitable Solution
1	High	Low	40	No
2	Medium	Medium	60	Yes
3	High	Medium	75	Yes
4	Low	High	65	Yes
5	Medium	High	80	Yes

Explain how selection chooses the best individuals, how crossover generates new offspring, and how mutation helps in exploring the hypothesis space. Also, provide the final optimized solution after evolution.

5. In the context of Neural Networks for classification, consider a dataset used to predict whether a patient has a disease based on attributes such as Age, Blood Pressure, Glucose Level, and Lifestyle Risk. The output indicates whether the disease is present (Yes) or not (No). Using this dataset, explain how a neural network model can be trained using backpropagation to classify the data. Also, evaluate the model performance by comparing actual and predicted outputs. (5 Marks)

Dataset:

Example	Age	Blood Pressure	Glucose Level	Lifestyle Risk	Actual Output	Predicted Output
1	45	High	High	High	Yes	Yes
2	50	Normal	High	Medium	Yes	No
3	30	Normal	Normal	Low	No	No
4	60	High	High	High	Yes	Yes
5	40	Normal	Normal	Medium	No	Yes

Using the above dataset, compute the values of True Positives, True Negatives, False Positives, and False Negatives. Based on these values, analyze the performance of the neural network using suitable evaluation metrics. Also, explain how preprocessing and feature selection improve the model's accuracy and reliability.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Question 1

Concept Involved

Functional blocks of a Machine Learning (ML) system – Data Collection, Preprocessing, Feature Extraction, Model Training and Evaluation; Supervised and Unsupervised Learning; Labelled and Unlabelled Data.

Definition

A machine learning system is a pipeline of connected blocks that converts raw data into a trained model which can make predictions or discover patterns on new data. In **supervised learning** the model learns from labelled data, where every input x has a known output y. In **unsupervised learning** the model learns from unlabelled data, where only x is available, and it finds hidden structure such as clusters on its own.

Scenario / Given Data

Approach	Scenario	Type of data
Supervised	Email spam filter for a mail service	Labelled: each email is tagged Spam (1) or Not spam (0)
Unsupervised	Customer segmentation for an online store	Unlabelled: customer records with no group tags

Answer (Step-by-Step Solution)

Step 1: Neat sketch of the functional blocks

Functional blocks of a Machine Learning System

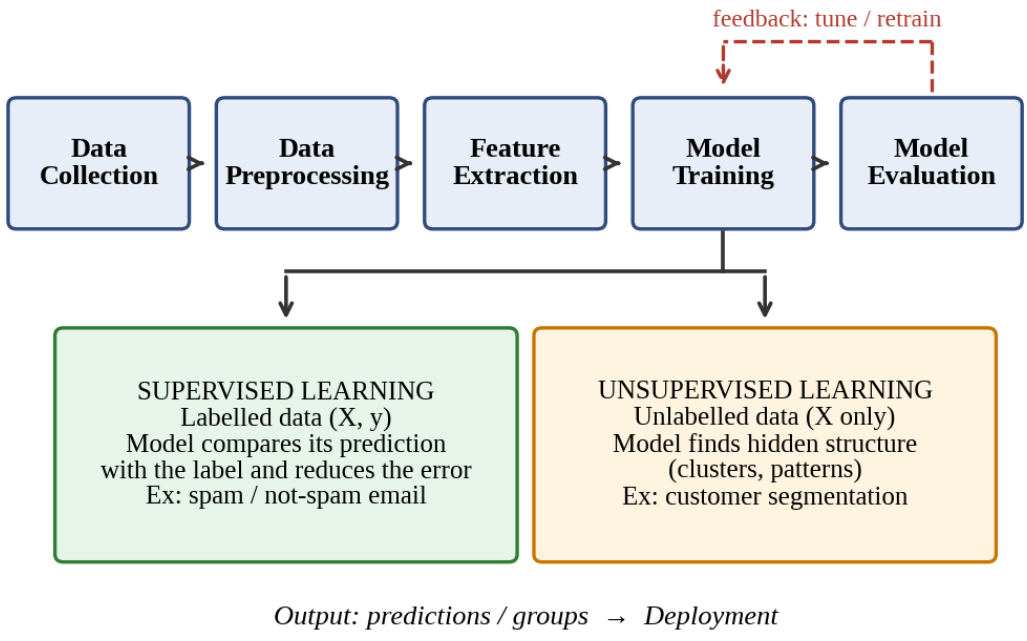


Fig. 1: Functional blocks of a machine learning system

Step 2: Explain each block and apply it to both examples

Block	Function	Supervised example (spam filter)	Unsupervised example (customer segmentation)
Data collection	Gather raw data from databases, sensors, logs or forms	10,000 emails, each labelled spam / not spam by users	5,000 customer records (age, annual income, spending score); no labels
Preprocessing	Clean, remove duplicates, fill missing values, normalise, split data	Remove HTML tags, lower-case, remove stop-words; split 80% train / 20% test	Fill missing income with the median; scale features to 0–1
Feature extraction	Convert raw data into numeric features that describe the problem	Word counts / TF-IDF, number of links, presence of "free"	Income and spending score (optionally reduced with PCA)
Model training	Learn model parameters from the features	Logistic regression / Naive Bayes trained on (x, y) pairs	K-means (K = 3) groups similar customers
Evaluation	Measure quality on data not used for training	Accuracy, precision, recall on labelled test set	Silhouette score, within-cluster distance, business review

Step 3: Handling labelled data (supervised learning)

Training data = $\{(x_i, y_i)\}$. The model predicts $\hat{y}$ for each x, and the label y supplies the correct answer.

Error = $y - \hat{y}$. For an email with 5 links and the word "free" ($y = 1$) the model gives $\hat{y} = 0.30$, so error = $1 - 0.30 = 0.70$.

The weights are adjusted (gradient descent) to reduce this error, and the process repeats over all emails for many epochs.

Finally the model is tested on held-out labelled emails by comparing predictions with the true labels.

Step 4: Handling unlabelled data (unsupervised learning)

Training data = $\{x_i\}$ only, so there is no label and no error signal. The model must group data by similarity.

Mini example (income in thousands, spending score): A(15, 80), B(16, 78), C(80, 20), D(85, 15). Start with centroids A and C.

Assign each point to its nearest centroid: B joins A (distance ≈ 2.2), D joins C (distance ≈ 7.1).

New centroids: Cluster 1 = (15.5, 79), Cluster 2 = (82.5, 17.5). Repeating gives no change, so the clusters are stable.

A human then interprets the groups: Cluster 1 = low income, high spenders; Cluster 2 = high income, low spenders.

Step 5: Compare the two approaches

Feature	Supervised learning	Unsupervised learning
Data	Labelled (x, y)	Unlabelled (x only)
Goal	Predict a known output	Discover hidden structure
Feedback	Error against the label	No label; similarity or density measure

Algorithms	Regression, decision trees, neural networks	K-means, hierarchical clustering, PCA
Evaluation	Accuracy, precision, recall, F1	Silhouette score, cluster validity

Application

The same pipeline is used in spam detection, medical diagnosis, credit scoring and speech recognition (supervised), and in customer segmentation, anomaly detection and recommendation systems (unsupervised). Every real project follows the same blocks, so a weak block (for example poor preprocessing) lowers the quality of the whole system.

Conclusion

A machine learning system works as a chain of five blocks: data collection, preprocessing, feature extraction, model training and evaluation, with feedback to improve the model. Supervised learning uses labelled data and learns by reducing the error against the label, as in the spam filter. Unsupervised learning uses unlabelled data and finds groups by similarity, as in customer segmentation.

Question 2

Concept Involved

Design of an ML system for disaster prediction; model selection (neural network vs other algorithms); preprocessing and feature selection; training, validation and evaluation; early warning and risk assessment.

Definition

A disaster-management ML system learns patterns from historical records and live sensor streams to estimate the probability or severity of a hazard before it strikes. Flood prediction is a **time-series forecasting / classification** problem (will the river level cross the danger mark in the next few hours?). Earthquake work is mainly **detection and early warning**: the system identifies an earthquake from the first (P) waves within seconds, estimates its size, and builds long-term hazard maps. Exact prediction of the time, place and magnitude of an earthquake is not possible with present science, so the system gives probabilities and seconds-to-minutes of warning rather than a firm forecast.

Scenario / Given Data

Item	Flood prediction	Earthquake early warning
Historical data	Daily/hourly rainfall, river discharge and level, soil moisture, past flood events, terrain and land use	Seismic catalogue (time, location, magnitude), fault maps, past ground-motion records
Real-time data	IoT river gauges, rain radar, satellite rainfall, dam release levels, weather forecasts	Seismometer waveforms, GPS ground displacement, accelerometers
Target	Water level in 6–24 h, or flood / no-flood	Earthquake detected? magnitude and shaking intensity
Challenge	Rare events, sensor gaps	Very rare events, need results in seconds

Answer (Step-by-Step Solution)

Step 1: System architecture

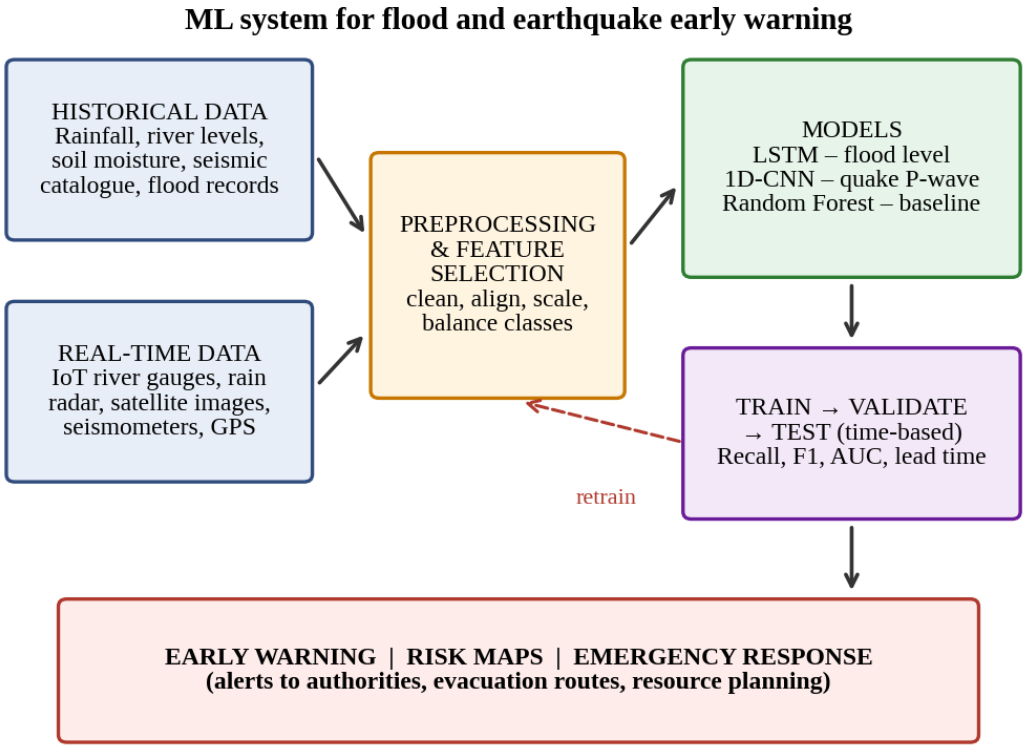


Fig. 2: ML system for flood and earthquake early warning

Step 2: Selection of the model

Algorithm	Best used for	Justification
LSTM / GRU (neural network)	Flood level forecasting from rainfall and river time series	Remembers long sequences and learns non-linear links such as rain → runoff delay → river rise
1D-CNN (neural network)	Detecting P-waves and estimating magnitude from seismic waveforms	Learns signal patterns automatically and runs fast enough for second-level warning
Random Forest / Gradient Boosting	Tabular flood-risk and hazard-zone classification; baseline model	Works with limited data, handles mixed features, gives feature importance and is easy to explain
Logistic regression	Simple baseline	Transparent; used to check that the complex model is really better

Is a neural network suitable? Yes, but not alone. Neural networks suit this task because the data are large, continuous and non-linear, and because time-series models (LSTM) and waveform models (CNN) capture patterns that simple models miss. Their drawbacks are the need for a lot of data, weaker interpretability, and poor learning of very rare events. Therefore a hybrid design is chosen: LSTM for flood forecasting, CNN for earthquake detection, and Random Forest for risk maps and as a baseline. The outputs can be combined (ensemble) to improve reliability.

Step 3: Data preprocessing

- **Cleaning:** remove sensor errors and impossible values (for example a negative water level); fill gaps by interpolation or the last valid value.

- **Time alignment:** resample all sources (hourly rain, 15-minute river level, satellite passes) to a common time step.
- **Normalisation:** min-max or z-score scaling so that rainfall (mm) and river level (m) are on the same scale.
- **Encoding and windowing:** encode categorical data such as land use; convert series into windows (for example the past 24 h → next 6 h) for the LSTM.
- **Class imbalance:** floods and earthquakes are rare, so use class weights or oversampling (SMOTE) to avoid a model that always predicts "no event".

Step 4: Feature selection

- **Correlation and mutual information:** keep features related to the target (for example 24-hour cumulative rainfall) and drop duplicates such as two highly correlated rain gauges.
- **Tree-based importance / Recursive Feature Elimination:** rank features using a Random Forest and remove the weakest.
- **PCA:** reduce many correlated satellite or waveform features to a few components.
- **Domain features:** rainfall over 1, 3, 6 and 24 h, upstream river level, soil saturation; for earthquakes the STA/LTA ratio, P-wave amplitude and dominant frequency.

Feature selection reduces noise and training time, lowers overfitting and improves accuracy.

Step 5: Training, validation and evaluation

Split by time, not randomly (for example 70% oldest data for training, 15% next for validation, 15% latest for testing), so the model never sees the future during training.

Training: the LSTM is trained with backpropagation through time, Adam optimiser and a weighted loss (MSE for level, cross-entropy for flood / no flood).

Validation: tune hyperparameters (layers, learning rate, window length) with time-series cross-validation; use early stopping when validation loss stops falling to prevent overfitting.

Testing: evaluate once on the untouched latest data, preferably including a past major flood event.

Metric	Why it matters here
Recall (sensitivity)	Most important: a missed disaster costs lives
Precision / false-alarm rate	Too many false alarms make people ignore warnings
F1-score, ROC-AUC	Balanced measure on imbalanced data
RMSE / MAE	Error of the predicted water level or magnitude
Lead time	How many minutes or hours of warning are given

Illustration: in a test period with 50 real flood events, the model warns for 45 and gives 8 false alarms. Recall = 45/50 = 90%, Precision = 45/(45 + 8) = 84.9%, F1 = 87.4%. Recall is high, so the model is suitable for warning use, with threshold tuning to cut false alarms.

Step 6: Deployment and feedback

Real-time data streams into the trained model, predictions refresh every few minutes, and alerts are pushed when the probability crosses a set threshold. Actual outcomes are logged and the model is retrained regularly.

Step 7: How the system helps authorities

- **Early warning:** forecasts river rise hours ahead and gives seconds of earthquake alert, so SMS, sirens and apps can warn people, and trains and gas lines can be shut down automatically.
- **Risk assessment:** produces flood-risk and hazard maps that show vulnerable areas, so authorities can plan drainage, building rules and insurance.
- **Emergency response:** supports evacuation routes, relief-camp locations, and the placement of boats, rescue teams and medical stock according to predicted severity.

Application

Such systems are used in national flood-forecasting centres, dam-operation planning, and earthquake early-warning networks, and they support smart-city disaster dashboards and insurance risk models.

Conclusion

A hybrid ML system is most suitable: an LSTM for flood forecasting, a CNN for earthquake detection and Random Forest for risk mapping. Careful preprocessing, feature selection, time-based training and validation, and recall-focused evaluation make it reliable. It gives authorities earlier warning, better risk maps and faster, better-targeted emergency response, although earthquakes can only be detected early or assessed probabilistically and not predicted exactly.

Question 3

Concept Involved

Multilayer Perceptron (MLP), Backpropagation Algorithm, Forward Pass, Error Calculation, Gradient-Descent Weight Update, Learning Rate and Activation Functions.

Definition

A multilayer perceptron is a feed-forward neural network with an input layer, one or more hidden layers and an output layer. **Backpropagation** is the algorithm that trains it. It first computes the output (forward pass), measures the error against the target, and then sends the error backwards using the chain rule to find how much each weight contributed to it. Each weight is then moved a small step against its gradient (gradient descent).

Neuron output: $o = \sigma(net)$, where $net = \Sigma\, w \cdot x$ and $\sigma(z) = 1 / (1 + e^{-z})$

Error: $e = t - o$

Output delta: $\delta_o = (t - o) \cdot o(1 - o)$

Hidden delta: $\delta_h = h(1 - h) \cdot v \cdot \delta_o$

Weight update: $w(new) = w(old) + \eta \cdot \delta \cdot$ (input to that weight)

Scenario / Given Data

A 2-2-1 MLP for binary classification (sigmoid in hidden and output layers; biases taken as 0 for simplicity).

Item	Value
Input	$x1 = 1, x2 = 0$
Input → hidden weights	$w11 = 0.5, w21 = 0.4$ (to h1); $w12 = 0.3, w22 = 0.6$ (to h2)
Hidden → output weights	$v1 = 0.7, v2 = 0.2$
Target / learning rate	$t = 1$ (class 1), $\eta = 0.5$

Answer (Step-by-Step Solution)

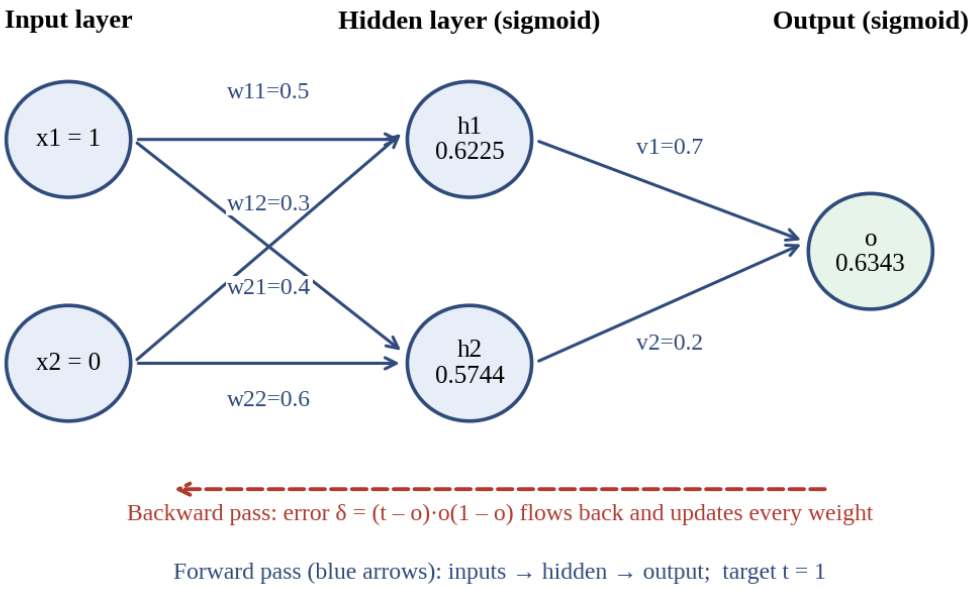


Fig. 3: MLP used for the worked example (forward and backward pass)

Step 1: Forward pass

net(h1) = (1 × 0.5) + (0 × 0.4) = 0.5 → h1 = σ(0.5) = 0.6225

net(h2) = (1 × 0.3) + (0 × 0.6) = 0.3 → h2 = σ(0.3) = 0.5744

net(o) = (0.7 × 0.6225) + (0.2 × 0.5744) = 0.4357 + 0.1149 = 0.5506

o = σ(0.5506) = 0.6343 (class 1 since o ≥ 0.5, but only with 63% confidence)

Step 2: Error calculation

e = t – o = 1 – 0.6343 = 0.3657

Squared error $E = \frac{1}{2}(t - o)^2 = \frac{1}{2}(0.3657)^2 = 0.0669$

Step 3: Backward propagation of error

Output delta: $\delta_o = 0.3657 \times 0.6343 \times (1 - 0.6343) = 0.0848$

Hidden delta h1: $\delta_1 = 0.6225 \times 0.3775 \times 0.7 \times 0.0848 = 0.01396$

Hidden delta h2: $\delta_2 = 0.5744 \times 0.4256 \times 0.2 \times 0.0848 = 0.00415$

Step 4: Weight update ($\Delta w = \eta \cdot \delta \cdot$ input)

Weight	Calculation	Δw	Old → New
v1	0.5 × 0.0848 × 0.6225	+0.0264	0.7000 → 0.7264
v2	0.5 × 0.0848 × 0.5744	+0.0244	0.2000 → 0.2244
w11	0.5 × 0.01396 × 1	+0.0070	0.5000 → 0.5070
w12	0.5 × 0.00415 × 1	+0.0021	0.3000 → 0.3021
w21, w22	input x2 = 0, so δ × 0 = 0	0	unchanged

Step 5: Check the improvement

With the new weights the output becomes o = 0.6416, which is closer to the target 1 than before (0.6343). The error falls from 0.3657 to 0.3584 after only one iteration; repeating over many examples and epochs drives the error towards zero.

Step 6: Effect of learning rate (η) on convergence

Learning rate	Effect on training	Result
Too small (for example 0.0001)	Tiny steps; training is very slow and may stall in a flat region	Slow convergence
Suitable (for example 0.01–0.5)	Steady decrease of error	Fast, stable convergence and good accuracy
Too large (for example > 1)	Steps overshoot the minimum; loss oscillates or diverges	No convergence, poor accuracy

Step 7: Effect of activation functions

Activation	Behaviour	Influence on learning
Sigmoid	Output 0–1; smooth	Good for binary output; derivative ≤ 0.25, so gradients vanish in deep networks and learning slows
Tanh	Output –1 to 1; zero-centred	Faster convergence than sigmoid, but still saturates
ReLU	max(0, z)	Gradient does not vanish for z > 0, so deep networks train faster; some neurons can "die"
Step / linear	Not differentiable / no non-linearity	Cannot be used for backpropagation in hidden layers

Non-linear activations are essential: without them the whole MLP collapses to a single linear model and cannot learn problems such as XOR.

Application

Backpropagation trains MLPs used for loan default prediction, disease classification, spam detection, image and speech recognition. In practice libraries use mini-batches, momentum or Adam, and a decaying learning rate to make convergence faster and more stable.

Conclusion

In the example, the forward pass gave o = 0.6343, the error was 0.3657, and backpropagation updated all weights so that the output improved to 0.6416 in one iteration. A suitable learning rate gives fast and stable convergence, and a non-linear activation (ReLU or sigmoid) lets the MLP learn complex decision boundaries and so reach higher accuracy.

Question 4

Concept Involved

Genetic Algorithm, Population, Chromosome Encoding, Fitness Function, Selection, Crossover, Mutation and Hypothesis Space Search.

Definition

A Genetic Algorithm (GA) searches the hypothesis space by evolving a population of candidate solutions. A **fitness function** scores each chromosome; **selection** keeps the fitter ones; **crossover** combines two parents to make offspring; and **mutation** randomly changes a gene to keep the search diverse. Over generations the average fitness rises and the population converges to a good solution.

Scenario / Given Data

Each candidate is a 4-bit chromosome made of two 2-bit genes: **Speed | Accuracy**, with Low = 00, Medium = 01, High = 10 (code 11 is unused). Total fitness = 40 + 60 + 75 + 65 + 80 = 320 and average fitness = 320 / 5 = 64.

Ex.	Speed	Accuracy	Chromosome	Fitness	Suitable	Selection probability
1	High	Low	10|00	40	No	40/320 = 12.5%
2	Medium	Medium	01|01	60	Yes	60/320 = 18.8%
3	High	Medium	10|01	75	Yes	75/320 = 23.4%
4	Low	High	00|10	65	Yes	65/320 = 20.3%
5	Medium	High	01|10	80	Yes	80/320 = 25.0%

*Assumption: the dataset gives fitness only for five combinations. For a new combination, fitness is estimated from the table: raising Accuracy from Medium to High adds +20 (60 → 80) and raising Speed from Medium to High adds +15 (60 → 75), so High-High ≈ 60 + 15 + 20 = 95. Marked with * below.*

Answer (Step-by-Step Solution)

Step 1: Generation 0 – Selection

The fittest individuals are Example 5 (80) and Example 3 (75). Roulette-wheel selection gives them the largest slices of the wheel (25.0% and 23.4%), while Example 1 (fitness 40, "No") has the smallest slice (12.5%) and is the first to be removed.

Selected parents: P1 = 01|10 (Medium, High, 80) and P2 = 10|01 (High, Medium, 75)

Step 2: Generation 0 → 1 – Crossover (single point after the 2nd bit)

P1 = 01 | 10 P2 = 10 | 01

Child 1 = 01 | 01 = (Medium, Medium), fitness 60

Child 2 = 10 | 10 = (High, High), fitness 95*

Child 2 takes the Speed gene of P2 and the Accuracy gene of P1, so it combines the best of both parents and is fitter than either of them.

Step 3: Generation 0 → 1 – Mutation

Mutation flips one bit with a small probability (for example 5% per bit). Example 4 = 00|10 (Low, High, 65) has its 2nd bit flipped: 00|10 → 01|10 = (Medium, High), fitness 80.

Mutation raised Speed from Low to Medium and fitness from 65 to 80. Mutation is random and can also lower fitness, so the rate is kept small and the best individuals are preserved. (A flip that creates the unused code 11 is rejected or repaired.)

Step 4: Form Generation 1 (the weakest individual, 10|00, is replaced)

Generation	Population (Speed|Accuracy)	Fitness values	Average	Best
0	10|00, 01|01, 10|01, 00|10, 01|10	40, 60, 75, 65, 80	64	80
1	01|10, 10|01, 10|10, 01|10, 01|01	80, 75, 95*, 80, 60	78	95*
2	10|10, 10|10, 01|10, 01|10, 10|01	95*, 95*, 80, 80, 75	85	95*

Step 5: Generation 1 → 2 – repeat selection, crossover and mutation

Selection picks 10|10 (95*), 01|10 (80) and 10|01 (75). Crossing 10|10 with 10|01 at the same point gives 10|10 and 10|01; crossing 10|10 with 01|10 gives copies of both. The weakest member 01|01 (60) is dropped.

Average fitness rises from 64 → 78 → 85, and the best fitness reaches 95*. Further generations only produce copies of 10|10, so the population has converged.

Step 6: Final optimised solution

Best chromosome = 10|10 → Speed = High, Accuracy = High, fitness ≈ 95*, Suitable = Yes

Among the five given examples the best was Example 5 (80). The GA found a still better, unseen solution (High, High) through crossover.

Step 7: Role of each operator

Operator	What it does	Effect on the search
Selection	Chooses parents in proportion to fitness (roulette wheel / top-k)	Passes good genes on; raises average fitness
Crossover	Swaps gene segments between two parents	Creates new combinations (exploitation)
Mutation	Randomly flips a bit with a small probability	Adds new genes; explores new regions of the hypothesis space; avoids premature convergence and local optima

Application

Genetic algorithms are used for parameter tuning of machine learning models, feature selection, scheduling, route planning, engineering design and neural-network architecture search, particularly where the function cannot be differentiated.

Conclusion

Starting from an average fitness of 64, selection, crossover and mutation raised the average to 85 in two generations and produced the optimal chromosome 10|10, i.e. **High Speed and High Accuracy (fitness ≈ 95, Suitable = Yes)**. Selection keeps good candidates, crossover recombines them, and mutation keeps exploring so the search does not get stuck.

Question 5

Concept Involved

Neural Network Classification, Backpropagation Training, Confusion Matrix (TP, TN, FP, FN), Performance Metrics, Preprocessing and Feature Selection.

Definition

A neural network classifier maps input attributes to a class (here Disease = Yes / No) through layers of weighted neurons. It is trained by backpropagation, which repeatedly reduces the difference between the actual and predicted output. A **confusion matrix** compares actual and predicted classes to measure performance. Taking **Yes (disease present) as the positive class**:

- **TP:** actual Yes, predicted Yes **TN:** actual No, predicted No

- **FP:** actual No, predicted Yes **FN:** actual Yes, predicted No

Given Data

Four input attributes, five patients (see the dataset in the question). Actual and predicted outputs are given for each patient.

Answer (Step-by-Step Solution)

Step 1: Prepare the data for the network (preprocessing)

Numeric input needs scaling and categories need encoding. Age is min-max scaled, Age′ = (Age – 30) / (60 – 30); Blood Pressure and Glucose: Normal = 0, High = 1; Lifestyle Risk: Low = 0, Medium = 0.5, High = 1; Output: Yes = 1, No = 0.

Ex.	Age′	BP	Glucose	Lifestyle	Target
1	0.50	1	1	1.0	1
2	0.67	0	1	0.5	1
3	0.00	0	0	0.0	0
4	1.00	1	1	1.0	1
5	0.33	0	0	0.5	0

Step 2: Network architecture

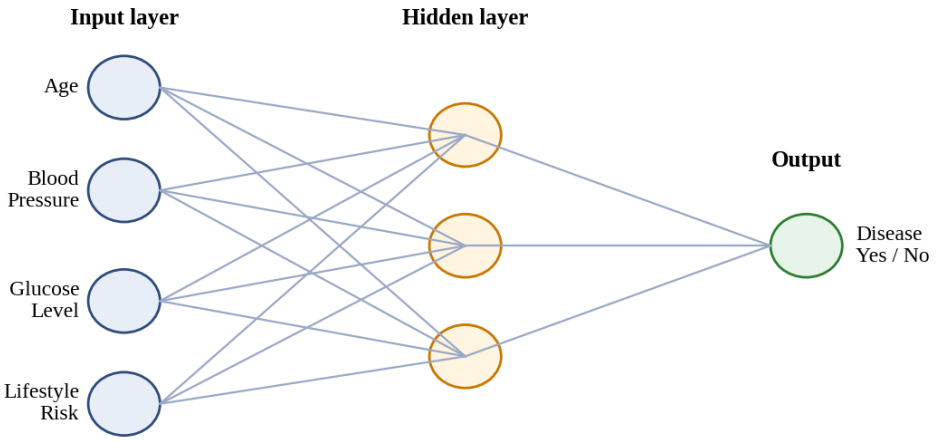


Fig. 4: Neural network used for disease classification (4 inputs, hidden layer, 1 output)

Four input neurons (one per attribute), a hidden layer (for example 3 neurons, ReLU or sigmoid) and one output neuron with sigmoid. Output ≥ 0.5 means "Yes".

Step 3: Training with backpropagation

1. Initialise the weights with small random values.
2. **Forward pass:** compute hidden activations and the output ŷ for a patient.
3. **Error:** compare ŷ with the target (cross-entropy or squared error), $E = \frac{1}{2}(t - \hat{y})^2$.
4. **Backward pass:** compute the output delta $\delta_o = (t - \hat{y}) \cdot \hat{y}(1 - \hat{y})$, propagate it to the hidden layer, $\delta_h = h(1 - h) \cdot v \cdot \delta_o$.
5. **Update:** $w(new) = w(old) + \eta \cdot \delta \cdot input$, for every weight.

6. Repeat for all patients (epochs) until the training error stops decreasing; use a validation set with early stopping to avoid overfitting.

Illustration (Example 2, wrongly classified): if the output is 0.40 against target 1, $\delta_o = (1 - 0.40) \times 0.40 \times 0.60 = 0.144$. For an output weight of 0.5 from a hidden neuron with activation 0.8 and $\eta = 0.5$, $\Delta w = 0.5 \times 0.144 \times 0.8 = 0.0576$, so the weight becomes 0.5576, which pushes the output towards "Yes".

Step 4: Compare actual and predicted outputs

Ex.	Actual	Predicted	Result
1	Yes	Yes	TP
2	Yes	No	FN
3	No	No	TN
4	Yes	Yes	TP
5	No	Yes	FP

	Predicted Yes	Predicted No	Row Total
Actual Yes	TP = 2	FN = 1	3
Actual No	FP = 1	TN = 1	2
Column Total	3	2	5

TP = 2, TN = 1, FP = 1, FN = 1

Step 5: Evaluation metrics

Metric	Formula	Value
Accuracy	(TP + TN) / Total = (2 + 1) / 5	60%
Error rate	(FP + FN) / Total = 2 / 5	40%
Precision	TP / (TP + FP) = 2 / 3	66.7%
Recall (Sensitivity)	TP / (TP + FN) = 2 / 3	66.7%
Specificity	TN / (TN + FP) = 1 / 2	50%
F1-score	2PR / (P + R) = 2(0.667)(0.667) / (1.333)	66.7%

Step 6: Analysis of the performance

- Accuracy is only 60%, so the model is moderately reliable at best.
- Recall is 66.7%: one of the three diseased patients (Example 2: age 50, high glucose, medium lifestyle risk) was missed. In diagnosis a false negative is the most serious error because treatment is delayed.
- Specificity is 50% and precision 66.7%: Example 5 was wrongly flagged, causing unnecessary tests and anxiety.
- With only five samples the figures are very unstable, so the model must be tested on a much larger dataset before any conclusion is final.

Step 7: How preprocessing and feature selection improve accuracy and reliability

Technique	What it does	Benefit to the model
Scaling / normalisation	Puts Age, BP, glucose on similar ranges (0–1)	Faster and more stable backpropagation; no feature dominates because of its size
Encoding of categories	Converts High / Normal / Low to numbers	Lets the network use categorical attributes
Handling missing values and outliers	Imputation, removal of impossible readings	Removes noise that causes wrong predictions
Class balancing	Class weights or oversampling	Improves recall for the minority (

Feature selection	Keeps relevant attributes using correlation, mutual information or importance scores	Reduces overfitting, training time and complexity, and improves generalisation

Application

Such neural-network classifiers are used for screening of diabetes, heart disease and cancer, and as decision-support tools that flag high-risk patients for further clinical tests.

Conclusion

The confusion matrix gives TP = 2, TN = 1, FP = 1 and FN = 1, so accuracy = 60%, precision = recall = F1 = 66.7% and specificity = 50%. The missed patient (FN) makes the model unsuitable for stand-alone diagnosis. Training with backpropagation, together with good preprocessing and feature selection, more data and class balancing, would raise recall and make the model more accurate and reliable.